

Chapter 4 过程模型Process Models

重点:

4.1.1 The Waterfall Model

4.1.2 Incremental Process Model

4.1.3 Evolutionary Process Model

process model为软件工程工作提供了特定的路线图。它定义了所有活动、行动和任务的流程、迭代的程度（迭代了几次，迭代的方式(可能是action,task等不断迭代)）、工作产品以及必须完成的工作的组织。

4.1 规定的过程模型Prescriptive Process Models

"prescriptive": 规定了一系列过程元素: framework activities, SE actions ,tasks ,work product, quality assurance , change control mechanisms

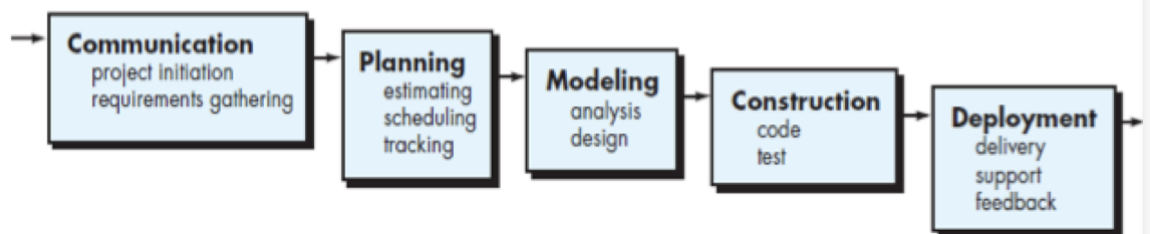
4.1.1 瀑布模型The Waterfall Model (也叫 classic life cycle/liner sequential)

适用场景: 当从沟通到部署都采用合理的线性工作方式时, 可以清楚地理解问题的需求; 也可以用在少数新的开发工作上。但是**需求必须是准确定义和相对稳定的**

特点:

1. 按照general/standard process**顺序开发**, **很少迭代**: planing是umbrella可以去掉
2. 过程明确, **需求清晰**
3. 不主张迭代, 但也有迭代
4. 若**开发时间长**, 用户需要很长时间才能得到成品
5. **会出现阻塞事件**: 某阶段受阻, 整个项目都会到blocking states, 也就是说, 没有形成milestone, 一个action, task卡住了就都卡了
6. 线性模型
7. 项目大的时候不要用, 适合需求清晰稳定的情况

FIGURE 4.1 The waterfall model *需求明确且稳定 → 很少进行迭代*



Planning: EST评估→计划→跟踪

瀑布模型的缺点:

waterfall models的问题:

- ①**实际项目很少遵循模型提出的顺序流程**。虽然线性模型可以适应迭代, 但它是间接这样做的。因此, 随着项目团队的推进, 变更可能会导致混乱。(项目发生变更时就不适用, 导致混乱)
- ②**客户通常很难明确地陈述所有要求**。瀑布模型需要这一点, 并且难以适应许多项目开始时存在的自然不确定性。(要求需求明确, 然而这点很难做到)

③ 客户要有耐心。程序的工作版本要到项目时间跨度的后期才可用。重大失误如果在审查工作计划之前未被发现，可能是灾难性的。(比如：waterfall会导致“阻塞状态”，在这种状态下，某些项目团队成员必须等待团队其他成员完成依赖任务。)(容错率低，且在项目后期发现难以修复，and阻塞状态)

4.1.1(extended)V-模型

V模型发现了软件开发和软件测试之间的映射关系（依赖关系）——>极大地推动了软件测试的发展

瀑布模型的变种 (与standard process非常相似)

瀑布模型对软件工程的最大贡献

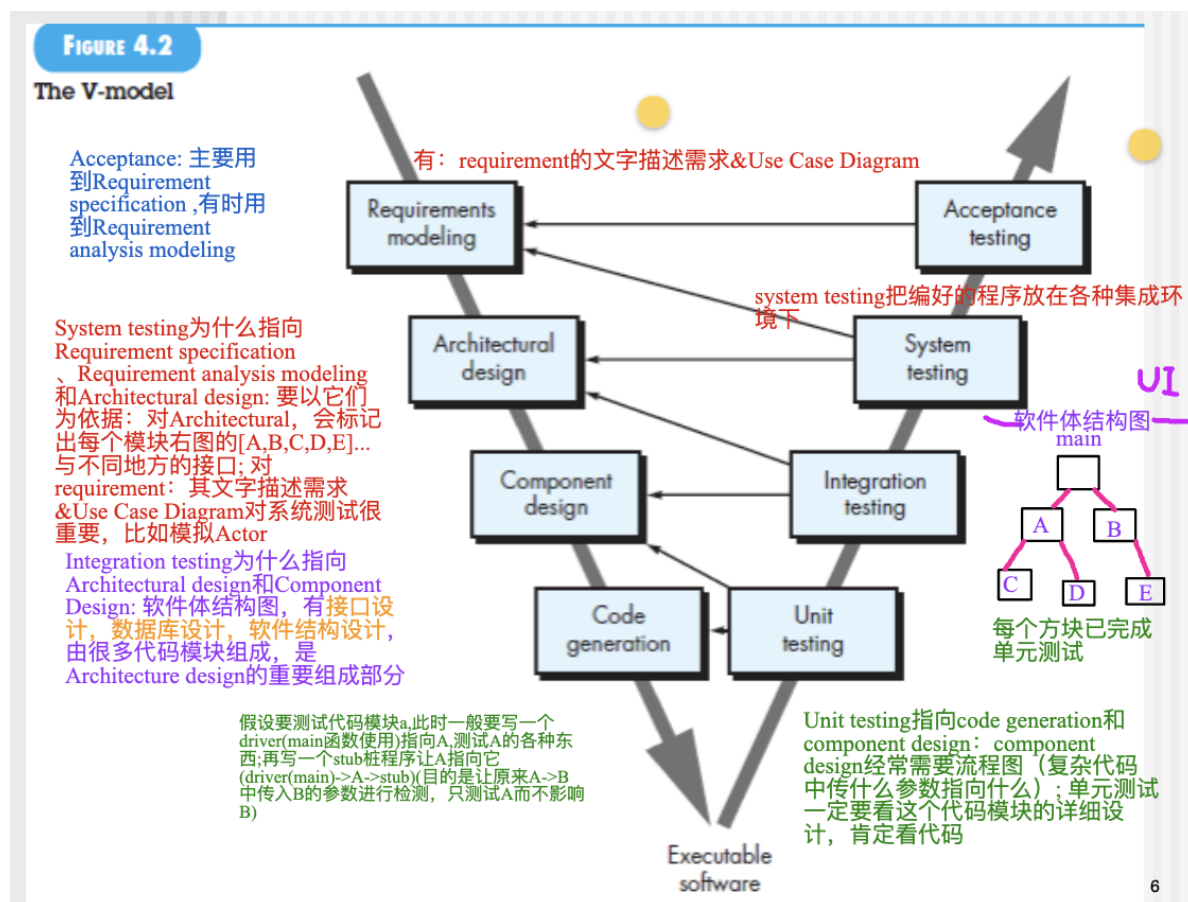
描述了质量保证行动与与沟通、建模和早期建设活动相关的行动之间的关系。

Unit Testing: 因为需要看底层代码，故依赖Code generation；因为复杂的代码中如何传参，参数指向什么很复杂，需要查看Component Design 和 详细设计规约

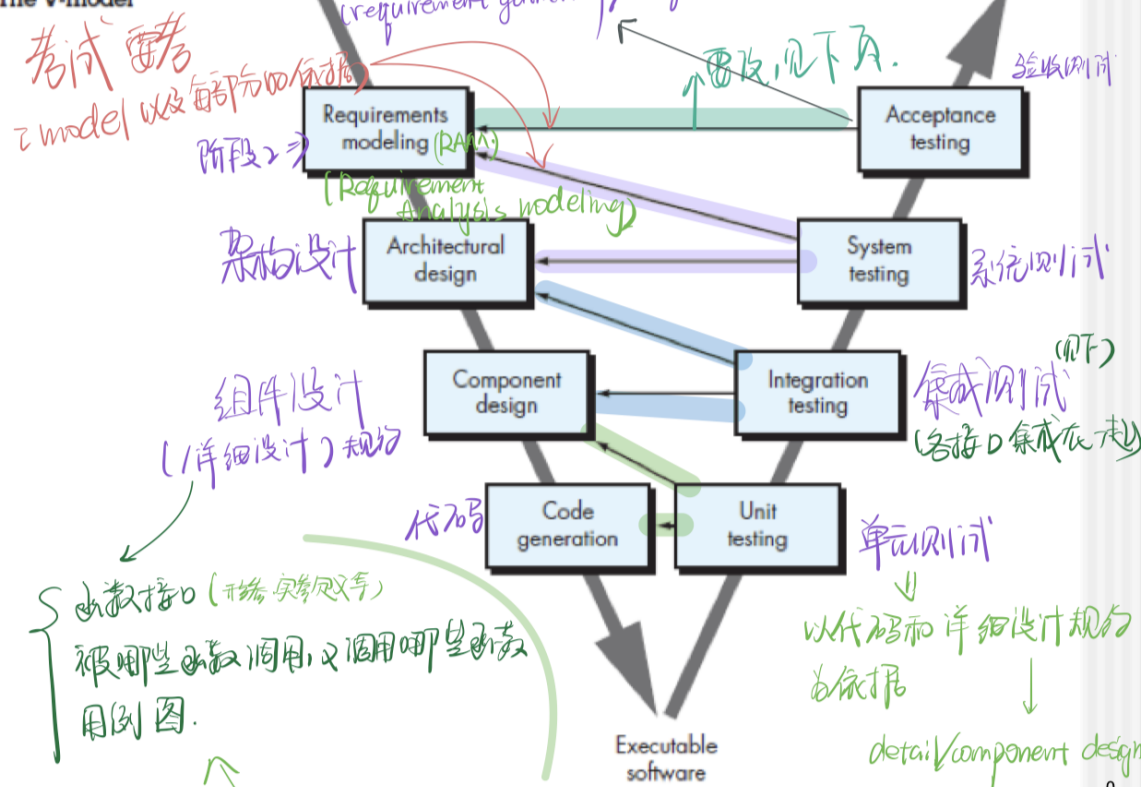
Integration Testing: 组件与组件的集成，看Component Design；还因为是和子系统之间的协作有关，涉及接口，数据库，软件结构设计，需要参考接口设计，数据设计，软件体系架构设计，故看Architectural Design

System Testing: 把编好的程序放在各种集成环境下；首先需要需求分析规约来进行模拟测试（主要看的是需求建模中的功能建模），比如模拟Actor，故需要Requirement Modeling；如果发现失败了，要检查什么接口有问题，故要参考Architectural Design(追溯子系统)

Acceptance Testing: 验收测试是面向用户的，用户不知道你的实现细节而是以需求为依据，所以是需求层面的（需要用到需求规约和需求分析规约），即Requirement Modeling——需求规约、需求分析规约

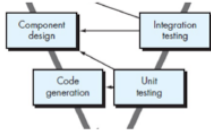


The V-model



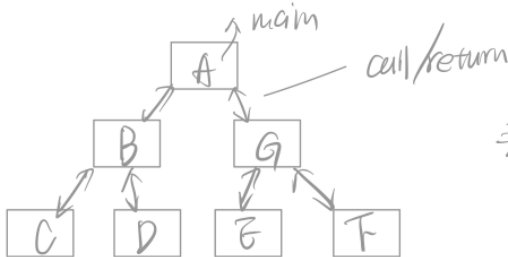
加:

集成测试以 detail / component design
和 Architectural design 为输入



架构设计包括3个部分
(第3张PPT笔记)

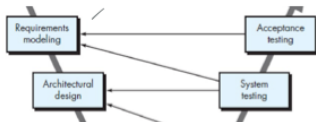
软件体系结构设计
数据库设计
接口设计 (interface)



→ 软件体系结构设计
怎么用, 哪些模块

知道了这些才可以
集成起来

System testing 系统测试以需求分析规范 requirement modeling
和 Architectural design 架构设计为输入



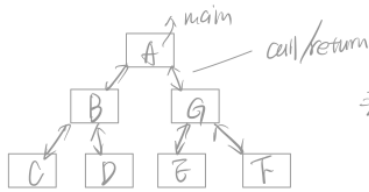
requirement modeling
需求建模

数据建模
功能建模
行为建模

→ 在系统测试中作为依据

做系统测试失败后要查哪个接口有问题

↓
要依据架构设计中系统内部结构图为依据



⇒ 附件结构设计
怎么调用，哪些模块

⇒ 调用关系：A→B→

→C→B→D

→B→B

→B→A

⇒ 根据这个去查系统测试
哪个模块出问题

Requirement specification

RS

RAM

依据

Acceptance
Testing

(Requirement Analysis modeling)

验收测试 ⇒ 业务

验收测试时客户需要以需求为依据

(与单元测试做的一样，区别是以需求为依据)

4.1.2 增量过程模型

场景：

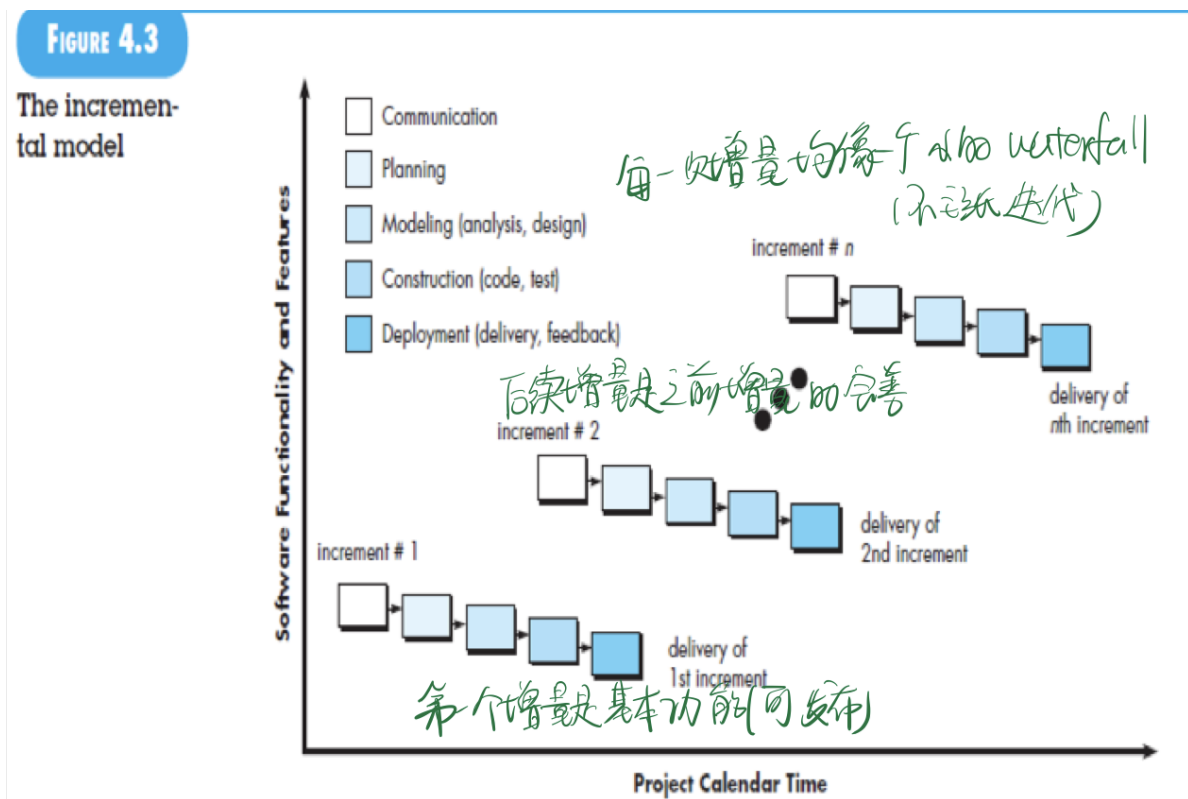
1. Non-linear 开发工作的整体范围排除了一个纯粹的线性过程。
2. Rapid Raw Product v1.0 可能迫切需要向用户快速提供一组有限的软件功能，然后在以后的软件版本中完善和扩展该功能。
3. Update v2.0++++ 有部分功能不成熟需要后期才增量补充；
4. Preview Function 市场要求先发布一部分功能，后续功能增量补充；
5. Parallel 一些内容交叉进行

结合了linear (waterfall模型) 和并行处理流parallel

当使用增量模型时，第一个增量通常是核心产品。也就是说，解决了基本要求，但许多补充功能（一些已知，另一些未知）仍未交付。核心产品由客户使用（或经过详细评估）。作为使用和/或评估的结果，为下一个增量制定计划。

特点：

- 每一次增量都生成可发布的产品increment product, **第一个增量形成核心基本产品**, 后续增量形成完善版本
- 每次增量是一个waterfall, 一次增量内不主张迭代
- 每次增量之间存在交叉和重复的情况
 - (比如: 第2个做coding时, 第3个可做需求建模) ——纵轴重合
- 后续增量是在前者基础上功能的完善
- 每次增量增加的功能需求是固定的?



4.1.3 演化过程模型Evolutionary Process Model

强调模型迭代iteration

场景:

1. Complex, change as development proceed 系统复杂, 模型演化需求变更;
2. limited version, 紧凑的市场期限要求一个功能有限的版本需要应对市场和企业的压力;
3. Requirement well understood, details yet to be defined 一系列核心产品和系统需求明确, 但产品和系统拓展的细节还没有被设计出来

两种常见的演化过程模型(都是迭代的):

4.1.3.1 原型模型Prototype

特点: 快速

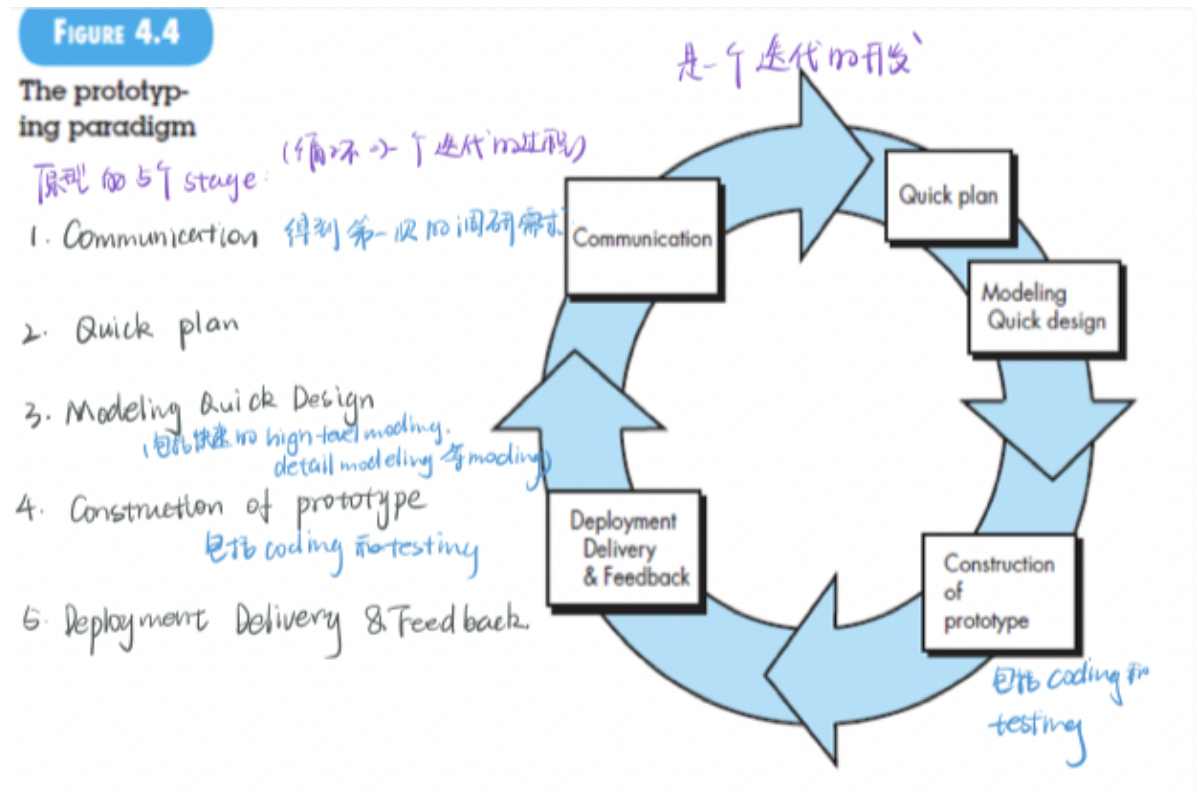
为什么要用Prototype(很可能考)

一般它是一个可以确定需求identifying software requirement的方法

描述: 客户定义了一组软件的一般目标但没有具体需求时/需求经常变化时; 开发者不知道具体开发细节时, 可帮助您和其他利益相关者更好地了解在需求模糊时要构建的内容。

迭代发生在原型被调整以满足不同利益相关者的需求时, 同时使您能够更好地了解需要做什么。【可以作为一种识别软件需求的工具identifying software requirements】

啥都快，不过Deployment模块貌似没有support环节？



Prototype的不足之处：

- （代码能跑了，但是static的没啥用）成型但不一定可靠：stakeholder看到的软件可运行，但不知道原型是固定在一起的，不知道在急于使其工作时**没有考虑整体软件质量或长期可维护性**。
- （代码都是垃圾的但能用，结果没改）错用：经常会在实现上做出妥协，以便让原型快速工作。不适当的操作系统或编程语言可能仅仅因为可用和已知而被使用；一个低效的算法可能会被简单地实现来展示能力。一段时间后，您可能会对这些选择感到满意，而**忘记了它们不合适的所有原因**。

Prototype主要是用来确定需求，真正开发时可能会被抛弃掉，换成更加注重质量的model；没有被抛弃的部分会成为真正系统的一部份

真正的开发要做到质量控制SQA

有大量外部接口的项目不适合用原型模型，例如云南婚姻项目（适合用螺旋模型）

4.1.3.2 螺旋模型Spiral Model

特点：适合**大型项目**，**风险较高的项目**，**需求不明确的项目**(big,risky,unclear)，将**原型设计的迭代性质 (iterative nature)**与**瀑布模型的受控和系统(controlled and systematic)**方面相结合。它为快速开发越来越完整的软件版本提供了潜力。

【是一种风险驱动模型，在planing阶段一定要做**risk analysis**】

FIGURE 4.5

运维阶段3个原因使功能修改和增加: 1.corrective: 纠错; 2. adaptive(适应性修改(eg:根据法规政策等修改) 3: enhanced: 新功能添加

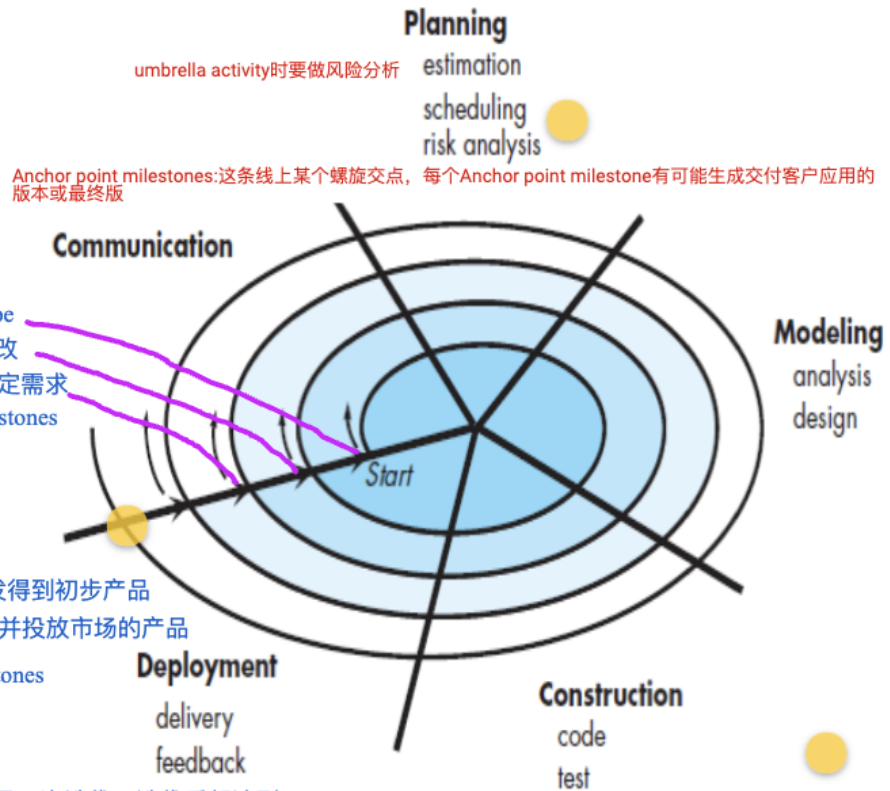
A typical spiral model

例子:

第一次迭代: prototype
第二次迭代: 讨论修改
第三次迭代: 基本确定需求
到达anchor point milestones
三次迭代后: 成功
从prototype得到需求 (包括function和 non-function)

第四次迭代: 增量开发得到初步产品
第n次迭代: 得到交付并投放市场的产品
到达anchor point milestones

运维阶段继续迭代:
运维阶段3个原因可以是一次迭代, 迭代后都达到anchor point milestones



解释:

每次迭代circuit后会得到一个work product

CPMCD:这5个步骤是可裁减的;

比如: 第一次迭代(circuit)主要为了得到需求规约, 则Communication后4个可以减去, 最后得到一个 Requirement Specification; 后面的迭代则都用到了后4个, 但详细程度重要程度也不同(比如:第二次迭代得到Prototype, 第三次可能会完成部分功能得到可发布的一个版本, 第四次进行功能完善等)

其他process models: 在软件交付时结束;

spiral model: 在整个软件的生命周期中都适用